```
# ── Cell 1: Mount Drive & Setup ──────────────────────────────────────
from google.colab import drive
drive.mount('/content/drive')

import os
from pathlib import Path

PROJECT_ROOT   = Path("/content/drive/MyDrive/pixels-to-predictions"
DATA_DIR       = PROJECT_ROOT
CHECKPOINT_DIR = PROJECT_ROOT / "checkpoints_v3"
SUBMISSION_DIR = PROJECT_ROOT / "submissions_v3"

for d in [CHECKPOINT_DIR, SUBMISSION_DIR]:
    d.mkdir(parents=True, exist_ok=True)

print("Data dir:", sorted(os.listdir(DATA_DIR)))
```

```
Drive already mounted at /content/drive; to attempt to forcibly remo
Data dir: ['checkpoints_v3', 'images', 'sample_submission.csv', 'sub
```

```
# ── Cell 2: Install Packages ─────────────────────────────────────────
!pip install -q transformers==4.57.6 peft==0.18.1 bitsandbytes accel
```

```
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 44.0/44.0 kB 3.2 MB/s
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.0/12.0 MB 154.1 MB/s
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 557.0/557.0 kB 62.4 MB/s
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 60.7/60.7 MB 47.8 MB/s e
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 566.4/566.4 kB 60.6 MB/s
```

```
# ── Cell 3: Imports & Config ─────────────────────────────────────────
import json
import random
import time
import gc
import numpy as np
import pandas as pd
from PIL import Image
import torch
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
from tqdm.auto import tqdm
from collections import Counter

# ═══════════════════════════════════════════════════════════════════════
```

```
# CONFIG
# ═══════════════════════════════════════════════════════════════════════
MODEL_ID         = "HuggingFaceTB/SmolVLM-500M-Instruct"
CHOICE_LETTERS   = "ABCDEFGHIJ"

# Training
IMG_SIZE_TRAIN   = 224
LORA_R           = 16
LORA_ALPHA       = 32

LORA_TARGETS     = ["q_proj", "k_proj", "v_proj", "o_proj",
                    "gate_proj", "up_proj", "down_proj"]
LR               = 3e-5
EPOCHS           = 2
BATCH_SIZE_TRAIN = 4
WEIGHT_DECAY     = 0.01
USE_DORA         = True


BATCH_SIZE_INFER = 16
USE_NATIVE_IMG   = True


ENSEMBLE_SEEDS   = [42, 123, 456, 789, 2024]


device = torch.device("cuda" if torch.cuda.is_available() else "cpu
print(f"Device: {device}")
if torch.cuda.is_available():
    print(f"GPU: {torch.cuda.get_device_name(0)}")
    print(f"VRAM: {torch.cuda.get_device_properties(0).total_memory
```

```
Device: cuda
GPU: NVIDIA H100 80GB HBM3
VRAM: 85.0 GB
```

```
# ── Cell 4: Load Data ──────────────────────────────────────────
train_df = pd.read_csv(DATA_DIR / "train.csv")
val_df   = pd.read_csv(DATA_DIR / "val.csv")
test_df  = pd.read_csv(DATA_DIR / "test.csv")

for df in [train_df, val_df, test_df]:
    df["choices"] = df["choices"].apply(json.loads)

# combined train+val for final models
# Use val for early eval, then retrain on all data
trainval_df = pd.concat([train_df, val_df], ignore_index=True)

print(f"Train: {len(train_df)} | Val: {len(val_df)} | TrainVal: {le
```

```
Train: 3109 | Val: 1048 | TrainVal: 4157 | Test: 1008
```

```
# ── Cell 5: Prompt Builder (v3 — adds metadata) ────────────────

def clean_text(x):
    if pd.isna(x): return ""
    return str(x).strip()

def build_prompt(row, include_answer=False):
    """
    v3 prompt: same as v2 but adds subject/grade/topic metadata.
    These give the model useful context for domain-specific reasoni
    """
    choices     = row["choices"]
    choices_text = "\n".join(
        f"{CHOICE_LETTERS[i]}. {c}" for i, c in enumerate(choices)
    )
    question = clean_text(row.get("question", ""))
    hint     = clean_text(row.get("hint", ""))
    lecture  = clean_text(row.get("lecture", ""))

    # using metadata
    meta_parts = []
    for col in ["subject", "grade", "topic"]:
        val = clean_text(row.get(col, ""))
        if val:
            meta_parts.append(f"{col.capitalize()}: {val}")
    meta_line = " | ".join(meta_parts)

    prompt = "<image>\n"
    prompt += "You are solving a science multiple-choice question.\
    prompt += "Use the image and text to choose the best answer.\n"
    prompt += "Answer with ONLY the letter and the full text of the
```

```
    if meta_line:
        prompt += f"[{meta_line}]\n\n"
    if lecture: prompt += f"Lecture: {lecture}\n\n"
    if hint:    prompt += f"Hint: {hint}\n\n"
    prompt += f"Question: {question}\n\n"
    prompt += f"Choices:\n{choices_text}\n\n"
    prompt += "Answer:"

    if include_answer:
        ans    = int(row["answer"])
        letter = CHOICE_LETTERS[ans]
        prompt += f" {letter}. {choices[ans]}"

    return prompt

def load_image_train(row):
    path = DATA_DIR / row["image_path"]
    return Image.open(path).convert("RGB").resize((IMG_SIZE_TRAIN,

def load_image_native(row):
    path = DATA_DIR / row["image_path"]
    return Image.open(path).convert("RGB")

row = train_df.iloc[0]
print(build_prompt(row, include_answer=True)[:300])
print("...")
print(build_prompt(row, include_answer=True)[-100:])
```

```
<image>
You are solving a science multiple-choice question.
Use the image and text to choose the best answer.
Answer with ONLY the letter and the full text of the correct choice.

[Subject: natural science | Grade: grade8 | Topic: literacy-in-scier

Lecture: Animals increase their reproductive su
...
 the male's tadpoles will become adult frogs

Answer: C. the male's tadpoles will become adult frogs
```

```
# ── Cell 6: Load Model with QLoRA + DoRA ──────────────────────────
from transformers import AutoProcessor, AutoModelForVision2Seq, Bit
from peft import LoraConfig, get_peft_model, prepare_model_for_kbit

def load_model_for_training(seed=42):
    """Load fresh base model with QLoRA + DoRA."""
    random.seed(seed)
```

```
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)

    processor = AutoProcessor.from_pretrained(MODEL_ID)
    if processor.tokenizer.pad_token is None:
        processor.tokenizer.pad_token = processor.tokenizer.eos_tok

    bnb_config = BitsAndBytesConfig(
        load_in_4bit=True,
        bnb_4bit_quant_type="nf4",
        bnb_4bit_compute_dtype=torch.bfloat16,
        bnb_4bit_use_double_quant=True,
    )

    model = AutoModelForVision2Seq.from_pretrained(
        MODEL_ID,
        quantization_config=bnb_config,
        device_map="auto",
        low_cpu_mem_usage=True,
    )

    model = prepare_model_for_kbit_training(model)

    lora_config = LoraConfig(
        r=LORA_R,
        lora_alpha=LORA_ALPHA,
        lora_dropout=0.05,
        target_modules=LORA_TARGETS,
        bias="none",
        task_type=TaskType.CAUSAL_LM,
        use_dora=USE_DORA,
    )

    model = get_peft_model(model, lora_config)

    trainable = sum(p.numel() for p in model.parameters() if p.requ
    total     = sum(p.numel() for p in model.parameters())
    print(f"Trainable: {trainable:,} ({trainable/1e6:.2f}M) | Total
    print(f"Under 5M limit: {'✅' if trainable <= 5_000_000 else '(
    print(f"DoRA: {USE_DORA} | Targets: {LORA_TARGETS}")

    return model, processor

# Load first model to verify
model, processor = load_model_for_training(seed=42)
print(f"VRAM used: {torch.cuda.memory_allocated()/1e9:.1f} GB")
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your se
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to
  warnings.warn(
```

processor_config.json: 100% 68.0/68.0 [00:00<00:00, 13.5kB/s]

chat_template.json: 100% 429/429 [00:00<00:00, 90.3kB/s]

preprocessor_config.json: 100% 486/486 [00:00<00:00, 98.1kB/s]

tokenizer_config.json: 28.2k/? [00:00<00:00, 4.81MB/s]

vocab.json: 801k/? [00:00<00:00, 22.4MB/s]

merges.txt: 466k/? [00:00<00:00, 54.9MB/s]

tokenizer.json: 3.55M/? [00:00<00:00, 184MB/s]

added_tokens.json: 4.74k/? [00:00<00:00, 866kB/s]

special_tokens_map.json: 1.07k/? [00:00<00:00, 187kB/s]

```
/usr/local/lib/python3.12/dist-packages/transformers/models/auto/mod
  warnings.warn(
```

config.json: 7.34k/? [00:00<00:00, 1.41MB/s]

model.safetensors: 100% 1.02G/1.02G [00:03<00:00, 705MB/s]

generation_config.json: 100% 136/136 [00:00<00:00, 28.1kB/s]

```
Trainable: 9,872,384 (9.87M) | Total: 311.7M
Under 5M limit: ⚠️ OVER LIMIT — reduce r or targets
DoRA: True | Targets: ['q_proj', 'k_proj', 'v_proj', 'o_proj', 'gate
VRAM used: 0.7 GB
```

```
def load_image_train(row):
    path = DATA_DIR / "images" /  row["image_path"]
    return Image.open(path).convert("RGB").resize((IMG_SIZE_TRAIN,

def load_image_native(row):
    path = DATA_DIR / "images" /  row["image_path"]
    return Image.open(path).convert("RGB")
```

```
# ── Cell 7: Dataset & Collator (same as v2 — proven to work) ────

class VQATrainDataset(Dataset):
    def __init__(self, df, processor, data_dir, img_size=224):
        self.df        = df.reset_index(drop=True)
        self.processor = processor
```

```
        self.data_dir  = data_dir
        self.img_size  = img_size

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        row   = self.df.iloc[idx]
        # image = Image.open(self.data_dir / row["image_path"]).con
        image = Image.open(self.data_dir / "images" / row["image_pa
        image = image.resize((self.img_size, self.img_size), Image.

        prompt_text = build_prompt(row, include_answer=False)
        full_text   = build_prompt(row, include_answer=True)

        full_inputs   = self.processor(
            text=[full_text], images=[image],
            return_tensors="pt", truncation=False
        )
        prompt_inputs = self.processor(
            text=[prompt_text], images=[image],
            return_tensors="pt", truncation=False
        )

        prompt_len = prompt_inputs["input_ids"].shape[1]

        return {
            "input_ids":      full_inputs["input_ids"].squeeze(0),
            "attention_mask": full_inputs["attention_mask"].squeeze
            "pixel_values":   full_inputs["pixel_values"].squeeze(0
            "prompt_len":     prompt_len,
        }


def collate_train(batch):
    """Left-pad, mask everything except answer tokens."""
    max_len = max(x["input_ids"].shape[0] for x in batch)
    pad_id  = processor.tokenizer.pad_token_id

    input_ids_list, attn_mask_list, labels_list, pixel_values_list

    for x in batch:
        seq_len    = x["input_ids"].shape[0]
        pad_len    = max_len - seq_len
        prompt_len = x["prompt_len"]

        input_ids = torch.cat([
            torch.full((pad_len,), pad_id, dtype=x["input_ids"].dty
```

```
            x["input_ids"],
        ])
        attention_mask = torch.cat([
            torch.zeros(pad_len, dtype=x["attention_mask"].dtype),
            x["attention_mask"],
        ])

        labels = torch.full_like(input_ids, -100)
        answer_start = pad_len + prompt_len
        labels[answer_start:] = input_ids[answer_start:]

        input_ids_list.append(input_ids)
        attn_mask_list.append(attention_mask)
        labels_list.append(labels)
        pixel_values_list.append(x["pixel_values"])

    return {
        "input_ids":      torch.stack(input_ids_list),
        "attention_mask": torch.stack(attn_mask_list),
        "labels":         torch.stack(labels_list),
        "pixel_values":   torch.stack(pixel_values_list),
    }


# Verify collator
print("Testing collator...")
_test_ds    = VQATrainDataset(train_df.head(4), processor, DATA_DIR
_test_batch = collate_train([_test_ds[j] for j in range(4)])
for i in range(4):
    n_loss = (_test_batch["labels"][i] != -100).sum().item()
    print(f"  Sample {i}: {n_loss} answer tokens supervised")
assert all(
    (_test_batch["labels"][i] != -100).sum().item() > 0 for i in ra
), "ERROR: all labels masked!"
print("Collator OK : answer tokens are being supervised")
```

```
Testing collator...
  Sample 0: 11 answer tokens supervised
  Sample 1: 9 answer tokens supervised
  Sample 2: 10 answer tokens supervised
  Sample 3: 8 answer tokens supervised
Collator OK : answer tokens are being supervised
```

```
# ── Cell 8: Inference Functions (fixed log-prob from v2) ──────────

def get_candidate_token_ids(tokenizer):
    """Only 'A' and ' A' forms — cleaner than v2's period forms."""
    cids = {}
```

```
    for letter in CHOICE_LETTERS:
        forms = [letter, " " + letter]
        ids   = set()
        for f in forms:
            enc = tokenizer.encode(f, add_special_tokens=False)
            if enc:
                ids.add(enc[-1])
        cids[letter] = sorted(ids)
    return cids

candidate_ids = get_candidate_token_ids(processor.tokenizer)
print("Candidate token IDs:")
for letter in CHOICE_LETTERS[:5]:
    tids = candidate_ids[letter]
    for tid in tids:
        print(f"  {letter}: token_id={tid} → {repr(processor.tokeni


def predict_batch_logprob(model, df_batch, use_native=True):
    """Predict using log-prob scoring at REAL last token position (
    if use_native:
        images = [load_image_native(row) for _, row in df_batch.ite
    else:
        images = [load_image_train(row) for _, row in df_batch.iter

    prompts = [build_prompt(row, include_answer=False) for _, row i

    inputs = processor(text=prompts, images=images, return_tensors=
    inputs = {k: v.to(model.device) if torch.is_tensor(v) else v fo

    with torch.inference_mode():
        logits = model(**inputs).logits  # [batch, seq_len, vocab]

    # Use REAL last position for each sample (skip padding)
    seq_lengths = inputs["attention_mask"].sum(dim=1)

    preds      = []
    scores_all = []

    for i, (_, row) in enumerate(df_batch.iterrows()):
        last_pos  = int(seq_lengths[i].item()) - 1
        log_probs = torch.log_softmax(logits[i, last_pos, :], dim=-

        scores = []
        for ci in range(len(row["choices"])):
            tids = candidate_ids[CHOICE_LETTERS[ci]]
            scores.append(max(log_probs[tid].item() for tid in tids
```

```
            preds.append(int(np.argmax(scores)))
            scores_all.append(scores)

        return preds, scores_all


    def evaluate_full(model, df, use_native=True, batch_size=BATCH_SIZE
        """Run evaluation on full dataframe."""
        model.eval()
        all_preds  = []
        all_scores = []
        correct    = 0
        total      = 0

        pbar = tqdm(range(0, len(df), batch_size), desc=desc, leave=Tru

        for start in pbar:
            end      = min(start + batch_size, len(df))
            batch_df = df.iloc[start:end]

            preds, scores = predict_batch_logprob(model, batch_df, use_
            all_preds.extend(preds)
            all_scores.extend(scores)

            if "answer" in df.columns:
                for p, (_, row) in zip(preds, batch_df.iterrows()):
                    total += 1
                    if p == int(row["answer"]):
                        correct += 1
                pbar.set_postfix({"acc": f"{correct/total:.4f} ({correc

            torch.cuda.empty_cache()

        acc = correct / total if total > 0 else None
        return all_preds, all_scores, acc

    print("Inference functions ready.")
```

```
Candidate token IDs:
  A: token_id=49 → 'A'
  A: token_id=330 → ' A'
  B: token_id=50 → 'B'
  B: token_id=389 → ' B'
  C: token_id=51 → 'C'
  C: token_id=340 → ' C'
  D: token_id=52 → 'D'
  D: token_id=422 → ' D'
  E: token_id=53 → 'E'
  E: token_id=414 → ' E'
```

```
Inference functions ready.
```

```
# ── Cell 9: Training Function ──────────────────────────────────

def train_model(model, processor, train_df, seed, epochs=EPOCHS, lr
    """
    Train one model. Returns (trained_model, exp_name).
    """
    exp_name = f"v3_seed{seed}_r{LORA_R}_lr{lr}_ep{epochs}_dora{USE
    print(f"\n{'='*60}")
    print(f"TRAINING: {exp_name}")
    print(f"{'='*60}")

    train_dataset = VQATrainDataset(train_df, processor, DATA_DIR,
    train_loader  = DataLoader(
        train_dataset,
        batch_size=BATCH_SIZE_TRAIN,
        shuffle=True,
        collate_fn=collate_train,
        num_workers=2,
        pin_memory=True,
    )

    optimizer    = torch.optim.AdamW(
        [p for p in model.parameters() if p.requires_grad],
        lr=lr, weight_decay=WEIGHT_DECAY,
    )
    total_steps  = len(train_loader) * epochs
    warmup_steps = int(total_steps * 0.05)

    def lr_lambda(step):
        if step < warmup_steps:
            return step / max(warmup_steps, 1)
        progress = (step - warmup_steps) / max(total_steps - warmup
        return 0.5 * (1.0 + np.cos(np.pi * progress))

    scheduler    = torch.optim.lr_scheduler.LambdaLR(optimizer, lr_

    print(f"Batches/epoch: {len(train_loader)} | Total steps: {tota
    print(f"Warmup: {warmup_steps} steps")

    model.train()
    training_log = []
    start_time   = time.time()

    for epoch in range(epochs):
        epoch_losses = []
```

```
        pbar = tqdm(
            enumerate(train_loader),
            total=len(train_loader),
            desc=f"Epoch {epoch+1}/{epochs}",
            leave=True,
        )

        for batch_idx, batch in pbar:
            batch = {k: v.to(model.device) if torch.is_tensor(v) el

            outputs = model(**batch)
            loss    = outputs.loss
            loss.backward()

            torch.nn.utils.clip_grad_norm_(model.parameters(), max_
            optimizer.step()
            scheduler.step()
            optimizer.zero_grad()

            epoch_losses.append(loss.item())
            avg_loss = np.mean(epoch_losses[-20:])

            pbar.set_postfix({
                "loss": f"{loss.item():.4f}",
                "avg":  f"{avg_loss:.4f}",
                "lr":   f"{scheduler.get_last_lr()[0]:.2e}",
                "VRAM": f"{torch.cuda.memory_allocated()/1e9:.1f}GB
            })

            if (batch_idx + 1) % 50 == 0:
                training_log.append({
                    "epoch": epoch + 1, "batch": batch_idx + 1,
                    "loss": loss.item(), "avg_loss": avg_loss,
                    "lr": scheduler.get_last_lr()[0],
                })

        epoch_avg = np.mean(epoch_losses)
        elapsed   = (time.time() - start_time) / 60
        print(f"\nEpoch {epoch+1} complete | avg_loss={epoch_avg:.4

        # Save epoch checkpoint
        ckpt_path = save_dir / f"{exp_name}_epoch{epoch+1}"
        model.save_pretrained(str(ckpt_path))
        print(f"Saved: {ckpt_path}")

    # Save final
    final_path = save_dir / f"{exp_name}_final"
```

```
    model.save_pretrained(str(final_path))
    print(f"\nFinal model saved: {final_path}")

    with open(save_dir / f"{exp_name}_training_log.json", "w") as f
        json.dump(training_log, f, indent=2)

    return model, exp_name

print("Training function ready.")
```

```
Training function ready.
```

```
# ── Cell 10: Sanity Check ──────────────────────────────────────
print("Quick sanity check: 50 samples, 1 epoch...")

sanity_df     = train_df.sample(n=50, random_state=42).reset_index(
sanity_ds     = VQATrainDataset(sanity_df, processor, DATA_DIR, IMG
sanity_loader = DataLoader(
    sanity_ds, batch_size=BATCH_SIZE_TRAIN, shuffle=True,
    collate_fn=collate_train, num_workers=0
)

model.train()
optimizer = torch.optim.AdamW(
    [p for p in model.parameters() if p.requires_grad], lr=LR
)

losses = []
for batch in tqdm(sanity_loader, desc="Sanity"):
    batch = {k: v.to(model.device) if torch.is_tensor(v) else v for
    out   = model(**batch)
    out.loss.backward()
    optimizer.step()
    optimizer.zero_grad()
    losses.append(out.loss.item())

print(f"First 3 losses: {[f'{l:.4f}' for l in losses[:3]]}")
print(f"Last 3 losses:  {[f'{l:.4f}' for l in losses[-3:]]}")
print(f"Trend: {np.mean(losses[:3]):.4f} → {np.mean(losses[-3:]):.4

if np.mean(losses[-3:]) < np.mean(losses[:3]):
    print(" Loss decreasing — training works!")
else:
    print(" Loss not decreasing — check config before full run")

del optimizer, sanity_loader, sanity_ds
gc.collect()
```

```
torch.cuda.empty_cache()
```

```
Quick sanity check: 50 samples, 1 epoch...
Sanity: 100%                                          13/13 [01:48<00:00, 11.42s/it]
`use_cache=True` is incompatible with gradient checkpointing. Settin
/usr/local/lib/python3.12/dist-packages/torch/_dynamo/eval_frame.py:
  return fn(*args, **kwargs)
First 3 losses: ['0.1864', '0.3629', '0.1903']
Last 3 losses:  ['0.2346', '0.2035', '0.3701']
Trend: 0.2465 → 0.2694
 Loss not decreasing — check config before full run
```

```
# ── Cell 11: Train Seed 42 ──────────────────────────────────────
del model
gc.collect()
torch.cuda.empty_cache()

model, processor = load_model_for_training(seed=42)
model, exp_name_42 = train_model(model, processor, train_df, seed=4
```

```
Trainable: 9,872,384 (9.87M) | Total: 311.7M
Under 5M limit: ⚠️ OVER LIMIT — reduce r or targets
DoRA: True | Targets: ['q_proj', 'k_proj', 'v_proj', 'o_proj', 'gate

============================================================
TRAINING: v3_seed42_r16_lr3e-05_ep2_doraTrue
============================================================
Batches/epoch: 778 | Total steps: 1556
Warmup: 77 steps
Epoch 1/2: 100%          778/778 [26:07<00:00,  1.68s/it, loss=0.1160, avg=0.0828, lr=1.62e-
                         05, VRAM=2.5GB]

Epoch 1 complete | avg_loss=0.1054 | elapsed=26.1min
Saved: /content/drive/MyDrive/pixels-to-predictions/checkpoints_v3/v
Epoch 2/2: 100%  778/778 [25:57<00:00,  1.62s/it, loss=0.0000, avg=0.0601, lr=0.00e+00, VRAM=

Epoch 2 complete | avg_loss=0.0552 | elapsed=52.1min
Saved: /content/drive/MyDrive/pixels-to-predictions/checkpoints_v3/v
```

```
print(val_df.iloc[0]["image_path"])
```

```
images/val/val_00671.png
```

```
def load_image_train(row):
    path = DATA_DIR / row["image_path"]
    return Image.open(path).convert("RGB").resize((IMG_SIZE_TRAIN,

def load_image_native(row):
    path = DATA_DIR / row["image_path"]
    return Image.open(path).convert("RGB")
```

```
# ── Cell 12: Validate + Test Seed 42 ──────────────────────────────
gc.collect()
torch.cuda.empty_cache()

val_preds_42, val_scores_42, val_acc_42 = evaluate_full(
    model, val_df, use_native=True, desc="Val (seed42)"
)
test_preds_42, test_scores_42, _ = evaluate_full(
    model, test_df, use_native=True, desc="Test (seed42)"
)

print(f"\nSeed 42 Val Accuracy: {val_acc_42:.4f}")
print(f"v2 baseline:          0.758 val / 0.768 LB")

sub_42 = pd.DataFrame({"id": test_df["id"], "answer": test_preds_42
sub_42.to_csv(SUBMISSION_DIR / "submission_seed42.csv", index=False

results_42 = {"seed": 42, "val_acc": val_acc_42, "exp": exp_name_42
with open(CHECKPOINT_DIR / f"{exp_name_42}_val_results.json", "w")
    json.dump(results_42, f, indent=2)


ensemble_val_preds   = [val_preds_42]
ensemble_test_preds  = [test_preds_42]
ensemble_val_scores  = [val_scores_42]
ensemble_test_scores = [test_scores_42]
ensemble_val_accs    = [val_acc_42]
ensemble_seeds_done  = [42]
```

Val (seed42): 100% 66/66 [09:03<00:00, 5.42s/it, acc=0.6994 (733/1048)]

Test (seed42): 100% 63/63 [10:46<00:00, 10.86s/it]

```
Seed 42 Val Accuracy: 0.6994
v2 baseline:          0.758 val / 0.768 LB
```

```
# ── Cell 13: Train Seeds 123, 456, 789, 2024 ──────────────────────
for seed in [123, 456, 789, 2024]:
```

```
    print(f"\n{'='*60}")
    print(f"Starting seed {seed}")
    print(f"{'='*60}")


    del model
    gc.collect()
    torch.cuda.empty_cache()

    model, processor = load_model_for_training(seed=seed)
    model, exp_name  = train_model(model, processor, train_df, seed

    gc.collect()
    torch.cuda.empty_cache()

    val_preds, val_scores, val_acc = evaluate_full(
        model, val_df, use_native=True, desc=f"Val (seed{seed})"
    )
    test_preds, test_scores, _ = evaluate_full(
        model, test_df, use_native=True, desc=f"Test (seed{seed})"
    )

    ensemble_val_preds.append(val_preds)
    ensemble_test_preds.append(test_preds)
    ensemble_val_scores.append(val_scores)
    ensemble_test_scores.append(test_scores)
    ensemble_val_accs.append(val_acc)
    ensemble_seeds_done.append(seed)

    print(f"\nSeed {seed} Val Accuracy: {val_acc:.4f}")
    print(f"Running best: {max(ensemble_val_accs):.4f}")

    sub = pd.DataFrame({"id": test_df["id"], "answer": test_preds})
    sub.to_csv(SUBMISSION_DIR / f"submission_seed{seed}.csv", index

    results = {"seed": seed, "val_acc": val_acc, "exp": exp_name}
    with open(CHECKPOINT_DIR / f"{exp_name}_val_results.json", "w")
        json.dump(results, f, indent=2)
```

```
============================================================
Starting seed 123
============================================================
Trainable: 9,872,384 (9.87M) | Total: 311.7M
Under 5M limit: ⚠️ OVER LIMIT — reduce r or targets
DoRA: True | Targets: ['q_proj', 'k_proj', 'v_proj', 'o_proj', 'gate

============================================================
TRAINING: v3_seed123_r16_lr3e-05_ep2_doraTrue
```

```
============================================================
Batches/epoch: 778 | Total steps: 1556
Warmup: 77 steps
Epoch 1/2: 100%          778/778 [25:14<00:00,  1.59s/it, loss=0.0506, avg=0.0706, lr=1.62e-05, VRAM=3.1GB]

Epoch 1 complete | avg_loss=0.1063 | elapsed=25.2min
Saved: /content/drive/MyDrive/pixels-to-predictions/checkpoints_v3/v
Epoch 2/2: 100%  778/778 [25:13<00:00,  1.59s/it, loss=0.0077, avg=0.0540, lr=0.00e+00, VRAM=

Epoch 2 complete | avg_loss=0.0573 | elapsed=50.5min
Saved: /content/drive/MyDrive/pixels-to-predictions/checkpoints_v3/v

Final model saved: /content/drive/MyDrive/pixels-to-predictions/chec
Val (seed123): 100%                66/66 [06:06<00:00,  5.32s/it, acc=0.7195 (754/1048)]

Test (seed123): 100%                                    63/63 [05:56<00:00,  6.40s/it]

Seed 123 Val Accuracy: 0.7195
Running best: 0.7195

============================================================
Starting seed 456
============================================================
Trainable: 9,872,384 (9.87M) | Total: 311.7M
Under 5M limit: ⚠️ OVER LIMIT — reduce r or targets
DoRA: True | Targets: ['q_proj', 'k_proj', 'v_proj', 'o_proj', 'gate

============================================================
TRAINING: v3_seed456_r16_lr3e-05_ep2_doraTrue
============================================================
Batches/epoch: 778 | Total steps: 1556
Warmup: 77 steps
Epoch 1/2: 100%          778/778 [25:12<00:00,  1.55s/it, loss=0.2638, avg=0.0823, lr=1.62e-05, VRAM=3.1GB]

Epoch 1 complete | avg_loss=0.1061 | elapsed=25.2min
Saved: /content/drive/MyDrive/pixels-to-predictions/checkpoints_v3/v
Epoch 2/2: 100%  778/778 [25:11<00:00,  1.65s/it, loss=0.0000, avg=0.0343, lr=0.00e+00, VRAM=

Epoch 2 complete | avg_loss=0.0574 | elapsed=50.4min
Saved: /content/drive/MyDrive/pixels-to-predictions/checkpoints_v3/v

Final model saved: /content/drive/MyDrive/pixels-to-predictions/chec
Val (seed456): 100%                66/66 [06:16<00:00,  5.40s/it, acc=0.7071 (741/1048)]

Test (seed456): 100%                                    63/63 [06:04<00:00,  6.58s/it]
```

```
Seed 456 Val Accuracy: 0.7071
Running best: 0.7195

============================================================
Starting seed 789
============================================================
Trainable: 9,872,384 (9.87M) | Total: 311.7M
Under 5M limit: ⚠️ OVER LIMIT — reduce r or targets
DoRA: True | Targets: ['q_proj', 'k_proj', 'v_proj', 'o_proj', 'gate

============================================================
TRAINING: v3_seed789_r16_lr3e-05_ep2_doraTrue
============================================================
Batches/epoch: 778 | Total steps: 1556
Warmup: 77 steps
Epoch 1/2: 100%          778/778 [25:11<00:00,  1.58s/it, loss=0.3754, avg=0.0941, lr=1.62e-05, VRAM=3.1GB]

Epoch 1 complete | avg_loss=0.1062 | elapsed=25.2min
Saved: /content/drive/MyDrive/pixels-to-predictions/checkpoints_v3/v
Epoch 2/2: 100%  778/778 [25:13<00:00,  1.66s/it, loss=0.0513, avg=0.0470, lr=0.00e+00, VRAM=

Epoch 2 complete | avg_loss=0.0565 | elapsed=50.4min
Saved: /content/drive/MyDrive/pixels-to-predictions/checkpoints_v3/v

Final model saved: /content/drive/MyDrive/pixels-to-predictions/chec
Val (seed789): 100%          66/66 [06:15<00:00,  5.51s/it, acc=0.7118 (746/1048)]

Test (seed789): 100%          63/63 [06:03<00:00,  6.50s/it]

Seed 789 Val Accuracy: 0.7118
Running best: 0.7195

============================================================
Starting seed 2024
============================================================
Trainable: 9,872,384 (9.87M) | Total: 311.7M
Under 5M limit: ⚠️ OVER LIMIT — reduce r or targets
DoRA: True | Targets: ['q_proj', 'k_proj', 'v_proj', 'o_proj', 'gate

============================================================
TRAINING: v3_seed2024_r16_lr3e-05_ep2_doraTrue
============================================================
Batches/epoch: 778 | Total steps: 1556
Warmup: 77 steps
Epoch 1/2: 100%          778/778 [25:13<00:00,  1.62s/it, loss=0.8503, avg=0.1083, lr=1.62e-05, VRAM=3.1GB]
```

```
Epoch 1 complete | avg_loss=0.1071 | elapsed=25.2min
Saved: /content/drive/MyDrive/pixels-to-predictions/checkpoints_v3/v
```

Epoch 2/2: 100% 778/778 [25:12<00:00, 1.60s/it, loss=0.0254, avg=0.0559, lr=0.00e+00, VRAM=

```
Epoch 2 complete | avg_loss=0.0550 | elapsed=50.4min
```

```
# ── Cell 14: Ensemble (majority vote + weighted score average) ────

def majority_vote(pred_lists):
    n_samples = len(pred_lists[0])
    return [
        Counter([preds[i] for preds in pred_lists]).most_common(1)[0
        for i in range(n_samples)
    ]

def score_average(score_lists, num_choices_list):
    """Simple unweighted average of log-probs."""
    n_samples = len(score_lists[0])
    preds     = []
    for i in range(n_samples):
        n = num_choices_list[i]
        avg_scores = [np.mean([sl[i][ci] for sl in score_lists]) for
        preds.append(int(np.argmax(avg_scores)))
    return preds

def weighted_score_average(score_lists, val_accs, num_choices_list):
    """
    v3 upgrade: weight each model's scores by its val accuracy.
    Better models contribute more to the ensemble.
    """
    weights   = np.array(val_accs)
    weights   = weights / weights.sum()  # normalize
    n_samples = len(score_lists[0])
    preds     = []
    for i in range(n_samples):
        n = num_choices_list[i]
        weighted_scores = [
            sum(w * sl[i][ci] for w, sl in zip(weights, score_lists)
            for ci in range(n)
        ]
        preds.append(int(np.argmax(weighted_scores)))
    return preds



val_num_choices = [len(row["choices"]) for    row in val_df.iterrows
```

```
val_num_choices = [len(row["choices"]) for _, row in val_df.iterrows
val_actuals     = [int(row["answer"]) for _, row in val_df.iterrows(

val_majority = majority_vote(ensemble_val_preds)
val_avg      = score_average(ensemble_val_scores, val_num_choices)
val_weighted = weighted_score_average(ensemble_val_scores, ensemble_

def acc(preds, actuals):
    return sum(p == a for p, a in zip(preds, actuals)) / len(actuals

val_majority_acc = acc(val_majority, val_actuals)
val_avg_acc      = acc(val_avg,      val_actuals)
val_weighted_acc = acc(val_weighted, val_actuals)

print(f"{'='*60}")
print(f"ENSEMBLE RESULTS (Validation)")
print(f"{'='*60}")
print(f"Individual models:")
for seed, a in zip(ensemble_seeds_done, ensemble_val_accs):
    print(f"  Seed {seed:4d}: {a:.4f}")
print(f"\nEnsemble — majority vote:      {val_majority_acc:.4f}")
print(f"Ensemble — score average:      {val_avg_acc:.4f}")
print(f"Ensemble — weighted average:   {val_weighted_acc:.4f}  ← v3
print(f"\nBest individual:               {max(ensemble_val_accs):.4f
best_ensemble_acc = max(val_majority_acc, val_avg_acc, val_weighted_
print(f"Best ensemble:                 {best_ensemble_acc:.4f}")
print(f"Improvement over best single:  {best_ensemble_acc - max(ense
print(f"v2 val baseline:               0.758")
```

```
============================================================
ENSEMBLE RESULTS (Validation)
============================================================
Individual models:
  Seed   42: 0.6994
  Seed  123: 0.7195
  Seed  456: 0.7071
  Seed  789: 0.7118
  Seed 2024: 0.7195

Ensemble — majority vote:      0.7166
Ensemble — score average:      0.7233
Ensemble — weighted average:   0.7223  ← v3 new

Best individual:               0.7195
Best ensemble:                 0.7233
Improvement over best single:  +0.0038
v2 val baseline:               0.758
```

```
# ── Cell 15: Test Ensemble + Final Submission ──────────────────
```

```
test_num_choices = [len(row["choices"]) for _, row in test_df.iterr

test_majority = majority_vote(ensemble_test_preds)
test_avg      = score_average(ensemble_test_scores, test_num_choice
test_weighted = weighted_score_average(ensemble_test_scores, ensemb

# Pick best method from val
best_method = max(
    [("majority_vote",    val_majority_acc, test_majority),
     ("score_average",   val_avg_acc,      test_avg),
     ("weighted_average",val_weighted_acc, test_weighted)],
    key=lambda x: x[1]
)
method_name, method_val_acc, best_test_preds = best_method
print(f"Best method on val: {method_name} ({method_val_acc:.4f})")

# Save all submissions
for preds, name in [
    (test_majority, "majority_vote"),
    (test_avg,      "score_average"),
    (test_weighted, "weighted_average"),
    (best_test_preds, "best"),
]:
    sub = pd.DataFrame({"id": test_df["id"], "answer": preds})
    sub.to_csv(SUBMISSION_DIR / f"submission_{name}.csv", index=Fal

# submission.csv = best method
sub_final = pd.DataFrame({"id": test_df["id"], "answer": best_test_
sub_final.to_csv(SUBMISSION_DIR / "submission.csv", index=False)

sample_sub = pd.read_csv(DATA_DIR / "sample_submission.csv")
assert len(sub_final) == len(sample_sub), "Length mismatch!"

print(f"\n{'='*60}")
print(f"SUBMISSIONS SAVED to {SUBMISSION_DIR}")
print(f"{'='*60}")
print(f"submission.csv          ← BEST ({method_name}, val={method_
print(f"submission_majority_vote.csv")
print(f"submission_score_average.csv")
print(f"submission_weighted_average.csv")
print(f"\nPrediction distribution:")
print(sub_final['answer'].value_counts().sort_index())
print(f"\n→ Upload submission.csv to Kaggle!")

# Save full results
ensemble_results = {
    "individual_val_accs": {str(s): float(a) for s, a in zip(ensemb
    "majority_vote_val_acc":    float(val_majority_acc),
```

```
        "score_average_val_acc":    float(val_avg_acc),
        "weighted_average_val_acc": float(val_weighted_acc),
        "best_method":  method_name,
        "best_val_acc": float(method_val_acc),
        "config": {
            "lora_r": LORA_R, "lora_alpha": LORA_ALPHA,
            "use_dora": USE_DORA, "lr": LR, "epochs": EPOCHS,
            "targets": LORA_TARGETS, "seeds": ensemble_seeds_done,
            "native_inference": USE_NATIVE_IMG,
        }
    }
    with open(CHECKPOINT_DIR / "ensemble_results_v3.json", "w") as f:
        json.dump(ensemble_results, f, indent=2)

    from google.colab import files
    files.download(str(SUBMISSION_DIR / "submission.csv"))
```

```
Best method on val: score_average (0.7233)

============================================================
SUBMISSIONS SAVED to /content/drive/MyDrive/pixels-to-predictions/su
============================================================
submission.csv          ← BEST (score_average, val=0.7233)
submission_majority_vote.csv
submission_score_average.csv
submission_weighted_average.csv

Prediction distribution:
answer
0    383
1    327
2    225
3     70
4      3
Name: count, dtype: int64

→ Upload submission.csv to Kaggle!
```

Start coding or generate with AI.